

Article

Design and Implementation of a Trusted Food Supply Chain Traceability System with Incentive Using Hyperledger Fabric

Zhiyang Zhou ¹, Yaokai Feng ^{2,*} and Kouichi Sakurai ²¹ Graduate School of Information Science and Electrical Engineering, Kyushu University, Fukuoka 819-0395, Japan; xjp616089@gmail.com² Faculty of Information Science and Electrical Engineering, Kyushu University, Fukuoka 819-0395, Japan; sakurai@inf.kyushu-u.ac.jp

* Correspondence: fengyk@ait.kyushu-u.ac.jp

Abstract

Effective supply chain traceability is indispensable for ensuring food safety, which is a significant social issue. Traditional traceability systems are mostly based on centralized databases, relying on a single entity or organization and facing problems such as insufficient transparency and the risk of data tampering. To address these issues, many studies have adopted blockchain technology, which offers advantages such as decentralization and immutability. However, challenges such as data credibility and insufficient protection of private data remain. This study proposes a multi-channel architecture based on Blockchain (Hyperledger Fabric in this study), in which data is partitioned and managed across dedicated channels to strengthen the protection of sensitive information. Furthermore, a trust and incentive design is implemented, featuring a trust-value calculation function and a reward–penalty mechanism that encourage participants to upload more truthful data and improve the reliability of data before it is recorded on the blockchain. In this paper, the design and implementation of the proposed system are explained in detail, and its performance is examined using Hyperledger Caliper, a blockchain performance benchmark framework. Functional evaluations indicate that the proposed system can be correctly implemented and that it correctly supports supply chain traceability, trust- and incentive-related, privacy protecting and other functions as designed, while performance evaluations indicate that it can maintain stable performance under higher workloads, suggesting that the proposed approach is practical and applicable to food supply chain traceability scenarios.

Keywords: Hyperledger Fabric; blockchain; supply chain traceability; decentralized system



Academic Editors: Ben Soh and Singara Singh Kasana

Received: 30 December 2025

Revised: 31 January 2026

Accepted: 3 February 2026

Published: 5 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

1.1. Conventional Food Supply Chain Traceability and Its Weaknesses

Food safety is a pressing global social concern. The food supply chain encompasses the entire journey of food, from agricultural production through processing, packaging, storage, transportation, distribution, and finally reaching consumers [1]. At each stage, critical data about product origin, quality, and handling practices must be properly managed and protected against tampering [2]. Effective supply chain traceability enables swift identification of contamination sources, reduces the scale and cost of recalls, and enhances accountability across stakeholders, thus supporting safer and more reliable food systems [2].

Traditional traceability systems in food supply chains are typically centralized, relying on individual organizations or authorities to maintain records [3]. This configuration often

leads to poor transparency, heightened risk of data tampering, fragmentation, and difficulty ensuring comprehensive end-to-end traceability [4]. Centralized data silos hinder information sharing and cross-verification among stakeholders, undermining the effectiveness of food safety management and eroding consumer confidence [5]. Consequently, there is a critical need to develop a secure, transparent, and decentralized supply chain traceability system that protects data privacy, guarantees data integrity, and reinforces trust among all participants [6]. Many studies have utilized blockchain technology to address these issues.

1.2. Blockchain Technology and Blockchain-Based Food Supply Chain Traceability

Blockchain is a distributed ledger system that was initially conceived to support Bitcoin, but it has rapidly evolved into a general-purpose technology with applications across many domains [7]. Unlike centralized databases, blockchain operates without a single controlling authority, relying instead on consensus among network participants to validate transactions [8]. Information is grouped into blocks, and each block is cryptographically connected to its predecessor, forming a chronological chain that cannot be altered retroactively [8]. This immutable and transparent structure ensures that once data is confirmed, it becomes a permanent part of the record, resistant to manipulation or deletion. Beyond cryptocurrencies, blockchain has been adopted for tasks such as supply chain monitoring, healthcare data management, and digital identity systems. Its key features—decentralization, security, and trust establishment among untrusted parties—make it particularly suitable for scenarios where data integrity and transparency are paramount [9].

Hyperledger Fabric is a permissioned blockchain framework hosted by the Linux Foundation, designed to provide modularity, scalability, and privacy for enterprise applications [10]. Unlike public blockchains, Fabric supports fine-grained access control, enabling organizations to define who can participate and what operations they can perform [10]. Its architecture makes it particularly suitable for food supply chain traceability, where multiple stakeholders require secure data sharing while protecting sensitive business information [11]. By offering decentralized consensus and customizable trust models, Fabric ensures data integrity and accountability across the entire supply chain [12].

Chaincode in Hyperledger Fabric serves as the smart contract layer, enabling developers to implement custom business logic that is executed by peer nodes [13]. Written in general-purpose languages such as Go, Java, or Node.js, chaincode governs how assets are created, transferred, or queried on the ledger. It can perform complex operations including validation rules, event recording, and access restrictions [10]. Once installed and instantiated, chaincode allows participants to interact with the blockchain through transactions that trigger predefined functions [10]. By automating business rules, chaincode reduces errors and enforces consistency across the network. In food supply chains, these functions support provenance queries and product information verification [14].

Hyperledger Fabric also introduces the concept of channels, which are subnets within the network that maintain their own ledgers [15]. Each channel restricts data visibility to its member organizations, allowing different groups of participants to transact privately while still benefiting from the shared infrastructure. Transactions in one channel are not accessible to members outside of it, thus ensuring confidentiality where required [15]. Channels can be created for bilateral agreements, consortium arrangements, or regulatory oversight. This mechanism provides a flexible balance between transparency and privacy [16]. In supply chain contexts, channels allow merchants to keep sensitive trade information confidential while still contributing to a trusted traceability system [17].

1.3. The Problems of Existing Blockchain-Based Food Supply Chain Traceability

One important issue in existing blockchain-based food supply chain traceability systems is data credibility [18]. Although blockchain ensures that once data is recorded on the ledger it cannot be altered, it cannot guarantee the authenticity of information before it is stored, false information may be written into the system. This limitation is called the “Garbage In, Garbage Out” (GIGO) problem [19], which means that if the input is low-quality or false data, then the output will also be low-quality or false data. In blockchain, if incorrect or poor-quality data is written onto the chain, it will be permanently preserved [19]. In supply chain traceability, the dangers of GIGO are particularly evident. For example, falsified origin data, adulterated product information, or manipulated quality records may mislead regulators and consumers [20]. Such issues weaken the reliability of the system, reduce stakeholder trust, and contradict the purpose of using blockchain to enhance transparency [21]. Even a small amount of false data can distort provenance tracking and hinder timely responses to food safety incidents. Therefore, measures must be taken to improve data reliability, such as introducing trust mechanisms, establishing reputation models, or integrating external verification technologies, in order to mitigate the risks posed by GIGO [22].

Data privacy is also a critical challenge in blockchain-based food supply chain traceability systems [6]. In practical supply chain environments, certain business information, such as internal production processes and raw quality inspection records, should not be disclosed to all participants. Protecting such sensitive data is essential, while their auditability must also be ensured to satisfy regulatory and accountability requirements [23]. Therefore, public blockchains, which provide full data transparency, are generally unsuitable for these application scenarios. For this reason, many existing studies adopt permissioned blockchains to restrict network participation. In addition, some works employ techniques such as Private Data Collections (PDC) [24] to achieve fine-grained access control, some introduce multi-chain architectures to store private data in sub-chains [23], and others adopt hybrid on-chain and off-chain storage approaches, where private data are stored off-chain and only their hashes or digests are recorded on-chain [25], thereby enhancing privacy protection while preserving auditability.

1.4. Motivation and Challenges

Increasing globalization, complex logistics networks, and rising consumer expectations for food safety and quality make transparency and reliability more critical than ever, which means that food supply chain traceability faces growing challenges and demands [26]. Traditional centralized systems struggle to handle these needs due to limited transparency, vulnerability to tampering, and lack of cross-organizational trust [2]. As a result, there is a pressing need for new technological solutions that ensure secure, transparent, and trustworthy management of supply chain data [6].

While blockchain provides significant advantages in terms of transparency and security, building a truly trustworthy and effective food supply chain traceability system still faces several challenges.

- **Data Privacy:** Although blockchain offers transparency, sensitive commercial information should not be openly visible to all participants. Achieving a balance between traceability and confidentiality remains a major concern [23].
- **Data Reliability:** The immutability of blockchain ensures data cannot be changed once recorded, but it cannot guarantee the trustworthiness of inputs before they are written. False or manipulated data may still be permanently stored, leading to GIGO problems [19].

- Performance: Supply chains involve high transaction volumes, and blockchain networks must maintain scalability and efficiency. Latency and throughput limitations can hinder real-time traceability in complex supply chains [27].

In light of the above challenges, this study is guided by the following research questions:

- RQ1: How to design an architecture based on Hyperledger Fabric to protect sensitive business data while still supporting cross-organizational food traceability?
- RQ2: How can the GIGO problem be mitigated and the reliability of traceability data be improved in blockchain-based food supply chain systems?
- RQ3: Can the proposed system maintain acceptable performance under realistic supply chain workloads?

The purpose of our research is to design and implement a food supply chain traceability system based on Hyperledger Fabric and to answer the 3 research questions. This system aims not only to protect sensitive and private data but also to improve the overall reliability and credibility of supply chain information.

The remainder of this paper is organized as follows. Section 2 reviews representative studies on blockchain-based supply chain traceability, clarifies the limitations of existing approaches in terms of privacy protection and data reliability and states contributions of our research. Section 3 presents the proposed system design, including the overall architecture, detailed system components, and chaincode design for traceability, cross-channel querying, and trust and incentive mechanisms, as well as practical implications for enterprises, consumers, and regulators. Section 4 describes the system implementation and functional testing results. Section 5 introduces the user interface developed for system demonstration. Section 6 reports the performance evaluation and analysis based on Hyperledger Caliper. Section 7 discusses the strengths and limitations of the proposed system and compares it with related works. Finally, Section 8 concludes the paper and outlines future work.

2. Literature Review and Our Contributions

2.1. Literature Review

IBM has launched a blockchain platform called IBM Food Trust [28], designed to enhance transparency, efficiency, and safety across the global food supply chain. Built on Hyperledger Fabric, it enables permissioned and tamper-resistant data sharing among growers, processors, retailers, and regulators. Its modular architecture—such as the Trace and Certifications modules—allows participants to access real-time provenance information, verify product authenticity, and accelerate recall processes. By allowing data owners full control over access permissions, the system establishes a trusted environment for sharing sensitive operational data. IBM Food Trust demonstrates how enterprise-grade blockchain systems can improve food freshness, reduce waste, and strengthen consumer confidence, making it one of the most influential industrial implementations of blockchain in food traceability. However, the system mainly relies on permissioned access control and enterprise governance, without introducing explicit mechanisms to mitigate the GIGO problem at the data input stage.

Marchese and Tomarchio [29] proposed a distributed agri-food supply chain traceability management system built on Hyperledger Fabric. The framework eliminates reliance on centralized authorities by providing a decentralized, tamper-resistant ledger for recording product lifecycle data. Smart contracts are employed to automate traceability logic, including rule enforcement and provenance tracking, ensuring consistency and compliance across stakeholders. The design also supports integration with scalable infrastructures such as cloud platforms, making it adaptable to complex agricultural networks and demonstrating its practical feasibility in real supply chain scenarios. However, this work focuses on

decentralized traceability but does not address data credibility issues, nor does it explore architectures for privacy protection.

Powell et al. [19] analyze the “garbage in, garbage out” problem in blockchain and IoT integration and propose several solutions to improve data integrity and trust in food supply chains. Drawing on the BeefLedger project for Australian beef exports, they argue that immutable ledgers alone cannot ensure data reliability. The authors introduce oracle identity and common knowledge mechanisms to enhance data validity and accountability, redefining blockchain’s role from a passive record of past events to an active tool for promoting reliable and trustworthy future states. However, although the GIGO problem is systematically analyzed, the study remains largely conceptual and does not propose a concrete solution that can be directly deployed.

Yang et al. [30] proposed a blockchain-based traceability system for fruit and vegetable agricultural products. Their framework addresses the shortcomings of centralized models, such as lack of transparency and unreliable records, by leveraging blockchain’s decentralization, immutability, and traceability features. A dual storage design, combining blockchain with a database, was introduced to reduce on-chain load and improve efficiency. The system also integrates cryptographic methods to protect sensitive information and employs a reputation-based smart contract mechanism to incentivize participants to provide authentic data, thereby improving overall trust in the supply chain. However, the proposed reputation mechanism is relatively coarse-grained, and the specific implementation details such as reputation calculation are not clearly specified.

Leteane and Ayalew [31] propose an extensible blockchain-based framework enhanced by a multi-trust-package trust model to improve the reliability of traceability data in food supply chains. Their architecture adds a trust-assessment layer before data recording, where smart contracts dynamically evaluate and score incoming inputs, thereby incentivizing participants to submit more reliable data to the ledger and largely mitigating the GIGO problem. The framework’s adaptability allows trust metrics and trust packages to be adjusted over time, supporting scalable deployment in complex food systems and strengthening the integrity and trustworthiness of traceability data. However, the framework requires trust score computation for each data submission, which introduces higher computational overhead and latency, while lacking a corresponding reward-and-penalty mechanism.

Sulyab et al. [32] propose a reward-based blockchain infrastructure designed to enhance provenance tracking and trust within the 3D IC supply chain. Their study introduces a reputation mechanism to evaluate participant behavior: when a product successfully passes verification, the corresponding entity receives additive reputation rewards, while defective or falsified data is detected, multiplicative reputation penalties are applied, and entities with excessively low reputation may even lose eligibility to participate in the supply chain. This “penalty-over-reward” design directly addresses the GIGO problem by significantly reducing unreliable inputs. In addition, the framework enables interoperability among multiple consortium blockchains, thereby improving data credibility across heterogeneous manufacturing environments. However, the scheme relies on specific application scenarios and employs a propagation-based reputation model, where updates to one organization’s reputation may trigger recalculation of related reputations, increasing computational overhead.

Ding et al. [23] proposed a permissioned blockchain-based double-layer framework for product traceability systems. The design introduces two distinct layers: a consortium blockchain for sharing traceability information with consumers and regulators, and a private blockchain inside enterprises for storing sensitive product data. This separation enables regulatory oversight while preserving enterprise privacy. The two layers are linked via hash pointers and APIs, separating data entry and querying to optimize performance.

Smart contracts manage interactions across layers, ensuring integrity and access control. This architecture enhances scalability, privacy, and regulatory inclusion, offering a refined model for enterprise-level traceability using blockchain. However, in the experimental evaluation, the study does not employ multi-channel architecture to physically isolate data between the main layer and the sub-layer but instead simulates the main layer and the sub-layer using different nodes within the same channel, which may affect the persuasiveness of the experimental results.

Liu et al. [33] propose a Hyperledger Fabric-based multi-channel architecture to enhance secure and efficient data exchange in the Internet of Vehicles (IoV). Their design isolates data into independent channels for different stakeholders—such as manufacturers, insurers, and traffic authorities—while enabling cross-channel interaction through chaincode. This structure ensures privacy and scalability by preventing unauthorized data access and supporting parallel data processing. By combining decentralized governance and inter-channel communication, the proposed system improves data integrity, trust, and interoperability across diverse IoV participants. However, although the study emphasizes cross-channel communication, the proposed mechanism is not fully elaborated, as it does not clearly specify how different organizations exchange data on the central channel.

To improve clarity and provide a structured overview of related works, Table 1 summarizes representative blockchain-based prior studies in terms of objectives, methodologies, key findings, and limitations.

Table 1. Comparison of Prior Studies.

	Objectives	Methodologies	Key Findings	Limitations
Study [28]	Improve food supply chain transparency	Permissioned blockchain (Fabric) platform	Improves traceability and recall efficiency	No explicit GIGO mitigation
Study [29]	Enable decentralized food traceability	Fabric with smart contracts	Ensures tamper-resistant lifecycle records	Ignores data credibility and privacy
Study [29]	Address the GIGO problem	Conceptual blockchain–IoT analysis	Shows immutability $\neq$ data reliability	No deployable system
Study [30]	Enhance trust in agri-food traceability	Blockchain + database with reputation	Reduces on-chain load and incentivizes honesty	Reputation design is coarse
Study [31]	Improve data reliability	Trust-based blockchain framework	Filters unreliable data before on-chain	High overhead, no incentives
Study [32]	Strengthen supply chain trust	Reward–penalty blockchain model	Discourages false data submission	Scenario-specific, high cost
Study [23]	Balance privacy and transparency	Double-layer permissioned blockchain	Separates public and private data	No physical multi-channel isolation
Study [33]	Secure multi-stakeholder data exchange	Fabric multi-channel architecture	Improves privacy and scalability	Vague cross-channel mechanism

In existing studies on blockchain-based supply chain traceability, some approaches introduce trust or incentive mechanisms to mitigate the GIGO problem, while others adopt multi-chain or multi-channel or other architectures to enhance privacy protection; however, each of these approaches still exhibits certain limitations. This study proposes a multi-channel architecture based on Hyperledger Fabric, in which data is partitioned and managed across dedicated channels, and compared with other multi-channel architectures,

it demonstrates clear advantages in cross-channel communication. Unlike multi-channel structures where channels are used solely for data isolation, this study not only protects sensitive data but also supports secure information exchange across channels. Compared with architectures that similarly introduce an intermediary channel for cross-channel communication, the proposed approach further enhances security and completeness by enforcing channel-level access control, thereby providing stronger privacy protection.

Furthermore, a trust score and incentive mechanism is implemented, which helps incentivize users to input truthful data and improve the reliability of data before it is recorded on the blockchain. In comparison with the incentive mechanisms proposed in other studies, the mechanism developed in this study achieves a balance between greater completeness and lightweight design. Some prior works rely solely on a simple buyer-rating system, whereas our approach incorporates both buyer evaluations and regulatory assessments, producing a more comprehensive and reliable trust score. In addition, the proposed model includes a more structured reward-and-penalty scheme that more effectively regulates participant behavior. On the other hand, several studies employ highly complex mathematical incentive models, which incur substantial computational overhead and are difficult to deploy in real-world blockchain environments. In contrast, our mechanism remains lightweight, incurs minimal computation cost, operates with stable execution performance, and is easier to deploy on a blockchain platform such as Hyperledger Fabric.

By combining the two, this study enhances both privacy and data reliability. The proposed system is implemented and tested, demonstrating stable performance and reliability.

2.2. Our Contributions

In this paper, a blockchain-based system architecture for food supply chain traceability using Hyperledger Fabric is proposed and its performance is also tested. Our main contributions are summarized as follows:

- **Multi-Channel Design:** We leverage the multi-channel mechanism of Hyperledger Fabric to separate merchants' private data into their own channels. Each merchant has control over access rights to its dedicated channel, ensuring that sensitive information is only shared with authorized participants. This design enhances privacy protection while still enabling collaboration within the overall traceability framework.
- **Trust Mechanism:** We actively utilize chaincode to implement a mechanism and introduce government authorities as supervisory organizations. By integrating the scores from regulators with evaluations from buyers, our system calculates comprehensive trust ratings for merchants. This incentive structure encourages merchants to upload authentic and reliable data, thereby greatly mitigating the GIGO problem.
- **Functional Testing:** We developed and deployed chaincode functions to test system operations such as product creation, sales, and provenance queries. The experimental results demonstrate that the proposed system is technically feasible and can effectively support food supply chain traceability in practice.
- **Performance Analysis:** We conducted performance evaluations under different workloads using Hyperledger Caliper. The results show that the system maintains stable throughput and low latency even under relatively high transaction loads, proving its robustness.

3. System Design

Proposed system establishes a blockchain-based food supply chain traceability framework. It integrates multi-channel data management, trust evaluation through smart contracts,

and privacy-preserving mechanisms. The design enhances the credibility of recorded information, protects sensitive data, and maintains transparent traceability across all participants.

3.1. System Architecture

As shown in Figure 1, our system consists of multiple organizations, peer nodes, orderer nodes and channels that collaboratively form a decentralized and trustworthy blockchain network for food supply chain traceability. Each organization (short for Org) maintains its own peer nodes, among which Org1, Org2, and Org3 have independent subchannels (channel B, C, and D) for managing private data, while all organizations share the main channel A for cross-organization data exchange. Orderer1 to Orderer3 form the ordering service responsible for consensus and block generation. The following subsections describe each component in detail.

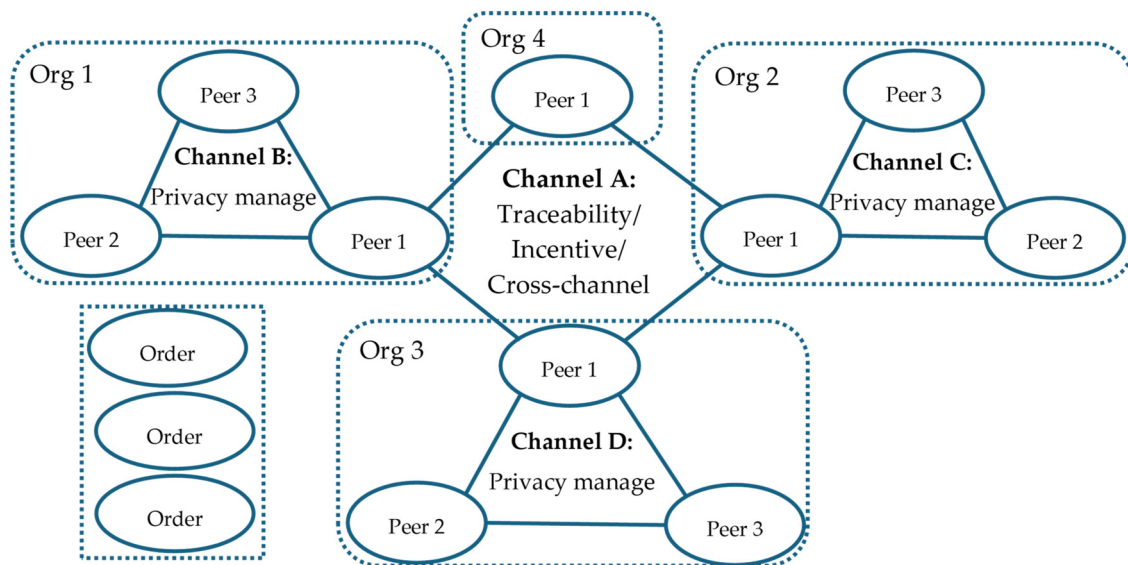


Figure 1. Structure of our proposal.

3.1.1. Organizations and Peer Nodes

In Hyperledger Fabric, organizations represent independent administrative domains that manage their own members, policies, and data within the network [9]. Each organization serves as a logical entity that reflects a real stakeholder in the system and can enforce access control and decision-making autonomy [9]. Our system simulates four organizations to represent a simplified food supply chain traceability network. In real-world deployments, the number of organizations could be much higher, and the model can be easily extended on top of our design. Specifically, Org1 is the producer, responsible for recording and managing product production data; Org2 is the processor, recording and managing processing information; Org3 is the retailer, responsible for recording and managing product sales information; and Org4 represents the government regulator, supervising enterprises. Peer nodes are the fundamental components of each organization, responsible for hosting ledgers, executing chaincode, endorsing transactions, and synchronizing state among network members [10]. Each organization can set multiple peer nodes for data sharing and storage. In our system, Org1, Org2, and Org3 each operate three peer nodes, while Org4 is assigned one peer node.

3.1.2. Orderer Nodes and Consensus

In Hyperledger Fabric, orderer nodes are responsible for establishing the total order of transactions, packaging them into blocks, and distributing these blocks to peer nodes

across the network [9]. They play a critical role in maintaining the consistency of the ledger by ensuring that all peer nodes receive transactions in the same sequence [9]. Our system sets three orderer nodes, namely Orderer1, Orderer2, and Orderer3. We adopt the etcd Raft consensus protocol, which is well-suited for enterprise blockchain applications where resilience and stability are important [34]. Etcd Raft works by electing a leader to coordinate ordering, replicating logs across follower nodes, and requiring majority agreement before a block is committed [34]. The leader receives transaction proposals, appends them to its log, and synchronizes entries with followers. Only when more than half of the nodes acknowledge the log entry is it considered committed, after which the block is finalized and distributed. If the leader fails, a new leader is elected automatically, ensuring continuity. This consensus mechanism provides strong consistency and rapid fault recovery [34]. Deploying three orderer nodes enhances system resilience by mitigating single-point failures and improving the fault tolerance of the network.

3.1.3. Channels

Channels in Hyperledger Fabric are private communication layers that allow a subset of organizations to share a dedicated ledger. They isolate data visibility and provide confidentiality while enabling collaboration among authorized members [9]. There are four channels in our system.

Channel A serves as the core channel of the entire system and includes Peer Node 1 from each organization. It is primarily responsible for storing and managing essential product traceability information as well as enterprise trust scores and related data. Channel A can also communicate with other channels, enabling organizations to invoke chaincode deployed on Channel A to access data maintained in other channels. This design ensures that key information remains centralized while still supporting cross-channel interoperability.

Channels B, C, and D are private internal channels for Org1, Org2, and Org3, respectively, with Peer Node 1, 2, and 3 of each organization participating. They store each enterprise's internal business details, raw quality inspection records, and other product data with commercial or privacy sensitivity. Access control mechanisms are enforced in every channel, ensuring that only authorized organizations can view or interact with the data contained within these private ledgers.

3.1.4. Certificate Authority (CA)

In Hyperledger Fabric, the Certificate Authority (CA) is a fundamental service responsible for managing digital identities in the permissioned blockchain network [35]. The CA issues cryptographic certificates to organizations, peer nodes, orderer nodes, and client applications, which serve as their unique identities within the system. These certificates are essential for authenticating participants, establishing secure communication, and enforcing access control policies [35]. The Fabric CA supports two main processes: registration, where administrators create new identities with defined roles and attributes, and enrollment, where users obtain signed certificates to join the network. It also provides certificate renewal and revocation services, ensuring that trust is continuously maintained throughout the lifecycle of participants [35]. By relying on public key infrastructure (PKI), the CA guarantees that every transaction is signed by an authorized entity [35]. This mechanism is vital for accountability, integrity, and privacy in a multi-organization blockchain network, where trust must be enforced through cryptography.

3.2. Chaincode

Chaincode in Hyperledger Fabric is the smart contract that defines business logic and governs interactions with the ledger [13]. It executes functions such as recording, querying,

and validating transactions. In our system, chaincode is written in Go language. Table 2 provides an overview of all chaincode functions in our system.

Table 2. Overview of the chaincode functions.

Channel	Module	Functions	Description
Channel A	Supplychain Traceability	CreateProduct(), ProcessMaterials(), SellProduct(), ReadProduct(), DeleteProduct(), GetLineage(), QueryProductCross()	These functions realize the core traceability logic of the food supply chain, enabling the recording and querying of production, processing, and sales events, as well as cross-channel provenance tracing.
	Supplychain Traceability	CreateProduct(), ProcessMaterials(), SellProduct(), ReadProduct(), DeleteProduct(), GetLineage(), QueryProductCross()	These functions realize the core traceability logic of the food supply chain, enabling the recording and querying of production, processing, and sales events, as well as cross-channel provenance tracing.
Channels B, C and D	Private Data Management	createProduct(), queryProduct(), updateProduct(), deleteProduct(), addAuthorizedCaller(), removeAuthorizedCaller(), getAuthorizedCallers(), checkAccess()	These functions manage private product data and access control within each organization's private channel, ensuring data separation and privacy protection.

As shown in Table 2, the chaincodes are mainly divided into two categories: those deployed on channel A and those deployed on channels B, C, and D. Chaincode deployed on channel A focuses on supply chain traceability and trust mechanisms, whereas chaincode deployed on channels B, C, and D emphasizes private data management and access control. The following subsections provide a detailed explanation of each chaincode function.

3.2.1. Chaincode on Channel A

The chaincode deployed on channel A implements both core functions of food supply chain traceability and a trust mechanism. The core functions include recording production, processing, and sales events, querying product status, deleting products and other functions. The trust mechanism consists of recording and querying enterprise trust scores as well as implementing incentive and penalty mechanisms. To support these operations, we designed separate variables for products, supply chain events, and enterprise scores, whose data structures are illustrated in Figures 2–4.

```

type Product struct {
    ID          string `json:"id"`
    Name        string `json:"name"`
    Manufacturer string `json:"manufacturer"`
    Owner       string `json:"owner"`
    Status      string `json:"status"`
}

```

Figure 2. Data structure of product.

```

type Event struct {
    EventID      string `json:"event_id"`
    InputProducts []string `json:"input_products"`
    OutputProducts []string `json:"output_products"`
    Actor        string `json:"actor"`
    EventType    string `json:"event_type"`
    Timestamp    string `json:"timestamp"`
}

```

Figure 3. Data structure of supply chain event.

```

type EnterpriseRating struct {
    Enterprise      string `json:"enterprise"`
    BuyerTotalScore int    `json:"buyer_total_score"`
    BuyerNum        int    `json:"buyer_num"`
    GovLatestScore  int    `json:"gov_latest_score"`
    CompositeScore  float64 `json:"composite_score"`
    IsBanned        bool   `json:"is_banned"`
    Grade           string `json:"grade"`
    Records         []ScoreRecord `json:"records"`
}

```

Figure 4. Data structure of enterprise score.

The core functions of supply chain traceability include recording production, processing, and sales events, querying product status, deleting products, performing provenance queries, and supporting cross-channel queries. Together, these functions create a transparent and auditable record of product lifecycles, ensuring that every stage of the supply chain can be verified. They enhance accountability among stakeholders, reduce disputes caused by inconsistent records, and provide reliable information for regulators and consumers, thereby strengthening trust and efficiency across the entire food supply chain. The following sections describe the detailed functionality of each chaincode function.

- **CreateProduct():** This function is used to record the creation of a new product in the blockchain ledger. Only Org1 is allowed to invoke this function, ensuring that production data is added exclusively by the producer. It first checks whether the product ID already exists, and if not, creates a new product with its details such as ID, name, and manufacturer. The function also records a corresponding production event with a timestamp, linking the product to the supply chain history.
- **ProcessMaterials():** This function records the processing stage in the supply chain. Only Org2 is allowed to invoke this function, ensuring that processing operations are performed exclusively by the processor. It first verifies the existence of all input raw materials, then creates new processed products with updated attributes such as ID, manufacturer, owner, and status. In addition, it generates a processing event with input and output product IDs, the actor, and a timestamp, thereby linking raw materials to processed products and extending the product's traceability history.
- **SellProduct():** This function records the sales stage of a product. Only Org3 is authorized to invoke this function, ensuring that sales operations are performed exclusively by the retailer. It first checks whether the product exists and confirms that its status is "Processed". If valid, the product ownership is transferred to the buyer, and the status is updated to "Sold". The function then creates a sales event containing the product ID, actor, and timestamp, which is stored on the ledger to complete the traceability record of the sales transaction.
- **ReadProduct():** This function is designed to retrieve the basic information of a product from the ledger using its unique ID. It first attempts to access the stored state; if the

product does not exist or an error occurs, it returns an appropriate error message. When data is successfully found, it is unmarshaled from JSON into a product object and returned to the caller. This function provides a straightforward way for stakeholders to query and verify product information within the blockchain system.

- **DeleteProduct():** This function is used to remove a product's basic information from the ledger by its unique ID. It first checks whether the product exists; if the product is not found or an error occurs during verification, an error message is returned. If the product does exist, the function deletes its state data from the state database. This allows outdated or invalid product records to be cleared when necessary, reducing storage pressure on the state database.
- **GetLineage():** This function is used to trace the provenance of a product throughout the supply chain. It collects transformation events related to the specified product and organizes them into a complete lineage record. The result shows all records of the product during production, processing, and distribution. This function enables stakeholders to verify the product's complete history and supports transparent and trustworthy traceability queries.
- **QueryProductCross():** This function provides cross-channel query capability in Hyperledger Fabric. It leverages the `InvokeChaincode` API [35], which allows a smart contract on one channel to invoke another chaincode function deployed on a different channel. The function constructs query arguments and calls the `QueryProduct()` function on the specified channel. If successful, it returns the product information obtained from the target channel. This mechanism enables seamless data access across multiple channels, ensuring that organizations can obtain product information beyond their own channel while preserving Fabric's channel-based privacy and isolation model.

To enhance the credibility of data recorded on the blockchain, our system implements a simple trust mechanism through chaincode. A government regulator is introduced as an organization within the blockchain to supervise enterprises, while both buyer ratings and regulator scores are combined to calculate comprehensive trust scores for enterprises. In addition, a basic incentive and penalty mechanism is included. This trust mechanism encourages enterprises to upload more authentic data, thereby mitigating the GIGO problem. The following sections describe the detailed functionality of each chaincode function.

- **RateByBuyer():** This function allows buyers to evaluate enterprises and record their ratings on the blockchain. After each real-world transaction, buyers can review the product information uploaded by the enterprise and then score the enterprise's credibility based on the received product. Each rating and its transaction timestamp are stored as part of the enterprise's rating records. The function also calculates the average score over the most recent seven days to reflect recent performance. It then combines the buyer's average score with the latest regulator score to compute a composite trust score for the enterprise. The formula is shown in Equation (1). *CompositeScore* is the composite trust score, *buyerAvg* is the buyer's average score, *govScore* is the regulator's score, and *weight* is a parameter determined by the regulator. If the regulator has not given a score for a long time, the weight can be reduced; otherwise, it can be increased. Finally, the updated rating is stored on the ledger, supporting incentive and penalty mechanisms to influence enterprise behavior.

$$\text{CompositeScore} = \text{buyerAvg} \times \text{weight} + \text{govScore} \times (1.0 - \text{weight}) \quad (1)$$

- **RateByGovernment():** This function enables the regulator to evaluate enterprises and record their scores on the blockchain. Only *Org4* is authorized to invoke this function, ensuring that regulatory ratings remain valid and credible. In general, the regulator conducts on-site inspections of the enterprise and then invokes this function. When

executed, the regulator's latest score is stored as part of the enterprise's rating data. The function then combines the regulator's score with the buyer's average score to generate a composite trust score for the enterprise. The calculation formula is the same as in the `RateByBuyer()` function. By updating this score on the ledger, the system ensures that both buyer evaluations and regulatory oversight are incorporated, providing a balanced and trustworthy measure of enterprise credibility.

- `updateGradeAndBan()`: This function is an auxiliary function invoked by other rating functions whenever an enterprise's score is updated. It automatically adjusts the enterprise's grade based on its composite score, assigning levels such as S, A, B, or C. In addition, if the score falls below a defined threshold, the enterprise will be marked as banned. Once banned, the enterprise loses permission to invoke chaincode functions, preventing untrustworthy participants from interacting with the blockchain system. This penalty mechanism reinforces data credibility and supports the overall trust framework.
- `QueryMerchantLatestScores()`: This function allows stakeholders to query the ratings and related information of an enterprise. It first retrieves the stored rating data from the state database, including the buyer's average score, the latest government score, and the enterprise's composite score. The function then returns a structured view containing the buyer's average score, the government score, the composite score, the enterprise's grade, and whether it is banned. This function provides a clear and reliable summary of the enterprise's trust status.
- `Ban()`: This function allows the regulator to explicitly ban an enterprise within the blockchain network. Only Org4 is authorized to invoke this function, ensuring that the banning decision is enforced by the regulator. When executed, the function retrieves the enterprise's rating record from the state database, updates its status by setting the `IsBanned` flag to true, and writes the result back to the ledger. In practice, this function is invoked in situations such as when the regulator discovers serious non-compliance during on-site inspections, or when the enterprise itself is involved in a major scandal and similar circumstances.
- `Unban()`: This function enables the regulator to lift the ban imposed on an enterprise. Only the government organization is authorized to invoke this function, ensuring that unbanning decisions remain under regulatory control. In practice, an enterprise that has been banned may file an appeal to the regulator. The regulator can then conduct offline inspections or adopt other measures, and based on the findings, decide whether to unban the enterprise. When invoked, the function retrieves the enterprise's rating record from the state database, sets the `IsBanned` flag to false, and writes the updated result back to the ledger, restoring the enterprise's ability to interact with the blockchain system.
- `ListSorted()`: This function retrieves all merchant ratings from the state database and returns them in descending order according to their composite scores. By doing so, it provides a ranked view of enterprises based on their trustworthiness and performance within the system. This function enables stakeholders to easily identify the most reliable enterprises. In future application-layer designs, the ranking results can be integrated into user interfaces, where enterprises with higher trust scores may receive greater visibility and recommendation. This approach serves as a form of reward for highly rated enterprises and further incentivizes honest behavior.

3.2.2. Chaincode on Channel B, C, D

Channels B, C, and D are private channels belonging to Org1, Org2, and Org3, respectively, and the chaincode on them can only be invoked by each organization. The

chaincode deployed on these three channels is identical. Its main functions include creating, updating, querying, and deleting private product information, as well as controlling access permissions for other organizations that request data from within the channel. Through this design, the system achieves separation of data and strengthens the protection of sensitive information, ensuring that each enterprise maintains autonomy over its private records while still participating in the overall traceability framework. The following sections describe the detailed functionality of each chaincode function.

- `createProduct()/queryProduct()/updateProduct()/deleteProduct()`: These functions provide basic CRUD operations for product records. The `createProduct()` function adds a new product to the state database with its ID, name, and privacy information. The `queryProduct()` function retrieves a product's data, but access is restricted by an access control check to ensure that only authorized organizations can view the information. The `updateProduct()` function allows modification of an existing product by overwriting its record. The `deleteProduct()` function removes a product entry from the state database by its ID. Together, these functions enable organizations to manage private product data securely and efficiently.
- `addAuthorizedCaller()/removeAuthorizedCaller()/getAuthorizedCallers()/checkAccess()`: These functions are used to manage access control for private channels. The `addAuthorizedCaller()` function adds a new organization, identified by its MSPID, to the access control list (ACL). If the ACL already exists, it is updated; otherwise, a new list is created. The `removeAuthorizedCaller()` function performs the opposite operation, removing the specified MSPID from the ACL to revoke its access rights. The `getAuthorizedCallers()` function retrieves the entire ACL from the state database, making it easy to see which organizations currently have access. The `checkAccess()` function checks whether an organization has access rights. It is typically invoked before an external organization attempts to access data in the channel; the function parses the caller's MSPID and compares it with the ACL stored in the state database, granting access only if the MSPID is explicitly listed. Together, these functions provide a flexible mechanism for dynamically controlling which organizations can access data in private channels, thereby strengthening privacy protection.

3.3. Practical Implications

To translate the proposed design into practical applications, this section discusses the system's concrete usage scenarios and practical implications in real-world supply chains from the perspectives of enterprises, consumers, and regulators.

For enterprises, once registered as authorized participants in the blockchain network, they can upload production, processing, logistics and other related data according to their roles, and query publicly available traceability records and trust scores of other participants. With explicit authorization from other participants, enterprises may also be granted controlled access to selected sensitive data for compliance verification or collaborative business purposes. The system continuously evaluates enterprises based on their historical data submission behavior, making data credibility visible across the network. In practice, this allows enterprises to assess potential partners, monitor upstream and downstream reliability, and enhance data credibility and verifiability in cross-organizational collaboration without exposing sensitive business information.

For consumers, the system enables convenient access to publicly available product traceability data and merchant trust scores through simple interactions such as QR code scanning, thereby significantly enhancing traceability transparency. By retrieving immutable on-chain traceability records together with aggregated trust indicators reflecting merchants' historical behavior, consumers can evaluate product provenance and enterprise

credibility, strengthen their trust in participating merchants, and make more informed purchasing decisions without imposing additional user burden.

For regulators, the proposed framework supports a hybrid supervision model that combines on-chain trust scores with off-chain inspections. Regulatory authorities can assign ratings based on inspection results, which are recorded on-chain and incorporated into trust score calculations. In cases of severe violations or persistently low credibility, regulators may revoke an enterprise's authorization to participate in the blockchain network according to predefined rules and restore participation rights after compliance is reestablished. By linking on-chain credibility indicators with real-world regulatory actions, the system enables continuous, auditable, and evidence-based supervision while avoiding excessive intervention in enterprise operations.

4. Implement and Functional Testing

4.1. Deployment

Before testing the chaincode functions, we carried out the system deployment, which mainly consisted of two parts: the deployment of the Hyperledger Fabric network and the deployment of the chaincode.

4.1.1. Deployment of Hyperledger Fabric Network

The deployment of a Hyperledger Fabric network mainly involves setting up cryptographic identities, defining network policies, and creating secure communication channels among organizations [35]. Table 3 provides a detailed explanation of the deployment process.

Table 3. Network Deployment Steps.

Step	Description	Purpose
Step 1. Generate cryptographic certificates	The cryptogen tool is used to generate cryptographic certificates for each organization, including peer nodes and orderer nodes.	These certificates act as secure digital identities, enabling authentication and encrypted communication across the network.
Step 2. Define network structure and policies	Use the configtxgen tool to define the network structure, including channel configurations and organizational policies. This step also generates the genesis block.	The genesis block initializes the ordering service and establishes governance rules.
Step 3. Create channels	The peer channel create command is executed to establish a new channel.	A channel provides a private ledger that is shared only by designated organizations, ensuring data segregation and confidentiality.
Step 4. Join peer nodes to channels	Each organization's peer nodes use the peer channel join command to connect to the channel.	Once joined, peer nodes can store the channel's ledger and participate in the consensus process.
Step 5. Update anchor peer nodes	The peer channel update command is used to update the channel configuration and designate certain peer nodes as anchor peer nodes.	Anchor peer nodes act as communication hubs, enabling efficient message exchange between organizations within the channel.

4.1.2. Deployment of Chaincode

The deployment of chaincode mainly involves packaging, installing, approving, and committing the chaincode before it can be used by organizations on a channel [35]. This process ensures that all participating organizations agree on the chaincode definition and that the chaincode is securely instantiated within the channel. Table 4 provides a detailed explanation of the deployment process.

Table 4. Chaincode Deployment Steps.

Step	Description	Purpose
Step 1. Package chaincode	The peer lifecycle chaincode package command is used to package the chaincode into a file.	This step prepares the chaincode for distribution and installation.
Step 2. Install chaincode	The peer lifecycle chaincode install command installs the packaged chaincode on the peer nodes of each organization that will participate in the channel.	This step ensures that all relevant peers have the chaincode ready for use.
Step 3. Approve chaincode definition	With the peer lifecycle chaincode approveformyorg command, each organization reviews and approves the chaincode definition, ensuring consensus before committing.	This guarantees organizational agreement on the chaincode definition before it is committed.
Step 4. Commit chaincode to channel	The peer lifecycle chaincode commit command commits the approved chaincode definition to the channel.	This makes the chaincode available for execution by all participating peer nodes.
Step 5. Test chaincode functions	After deployment, functions are invoked to test the chaincode's capabilities.	This confirms that the chaincode performs as expected and that the system operates correctly.

4.2. Functional Testing

After completing the system deployment, we conducted functional testing of the chaincode to verify that its core features worked as expected. We tested the functionality of all chaincode functions and verified their effectiveness. The following are the test results of several key functions.

Figure 5 illustrates the input parameters and the test result of invoking the CreateProduct() function. As shown in the figure, the invoked function is CreateProduct(), where the parameters “milk1”, “FreshMilk”, and “DairyFarm” represent the created product ID, product name, and manufacturer, respectively. The figure also shows that the function was successfully executed. This figure demonstrates that the system can correctly receive and process product creation requests, ensuring the effectiveness and traceability of the chaincode in the initial product creation stage.

```
c ' {"A" : -c ' {"Args":["CreateProduct","milk1","FreshMilk","DairyFarm"]} ' \
> --waitForEvent \
erAddre> --peerAddresses localhost:7051 \
--tlsR> --tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
> --peerAddresses localhost:9051 \
-tlsRoo> --tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
2025-11-04 20:03:36.314 JST 0001 INFO [chaincodeCmd] ClientWait -> txid [85c9115650b3078045596f4d1d3c5ca970109a7ddaff7c0811935cfb160e687b] committed with status (VALID) at localhost:7051
2025-11-04 20:03:36.318 JST 0002 INFO [chaincodeCmd] ClientWait -> txid [85c9115650b3078045596f4d1d3c5ca970109a7ddaff7c0811935cfb160e687b] committed with status (VALID) at localhost:9051
2025-11-04 20:03:36.318 JST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful
. result: status:200
```

Figure 5. CreateProduct() test result.

Figure 6 illustrates the input parameters and the test result of invoking the ProcessMaterials() function. As shown in the figure, the invoked function is ProcessMaterials(), where the parameters “milk1” and “cheese1” both represent product IDs, indicating that product milk1 has been processed into cheese1. The figure also shows that the function was successfully executed. This figure demonstrates that the system can correctly record product transformation events, ensuring that processing activities are accurately captured and that traceability is preserved throughout the supply chain.

```

> -c '{"Args":["ProcessMaterials","\\"milk1\\"","\\"cheese1\\""]}' \
> --waitForEvent \
> --peerAddresses localhost:9051 \
--tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
2025-11-04 20:10:33.368 JST 0001 INFO [chaincodeCmd] ClientWait -> txid [4f2d76fd1e29bb29b136f47fbc967a015e8f338c88498e84bcb69951e8ff8f3] committed with status (VALID) at localhost:9051
2025-11-04 20:10:33.373 JST 0002 INFO [chaincodeCmd] ClientWait -> txid [4f2d76fd1e29bb29b136f47fbc967a015e8f338c88498e84bcb69951e8ff8f3] committed with status (VALID) at localhost:7051
2025-11-04 20:10:33.373 JST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

```

Figure 6. ProcessMaterials() test result.

Figure 7 illustrates the input parameters and the test result of invoking the GetLineage() function. As shown in the figure, the invoked function is GetLineage(), where the parameter “cheese1” indicates that the product ID being queried for traceability is cheese1. The figure also shows the provenance query results, which record in detail the production, processing events, raw materials and other traceability information of this product. This figure demonstrates that the system can successfully reconstruct the history of a product, validates the successful execution of the CreateProduct() and ProcessMaterials() functions and confirms the transparency and reliability of supply chain traceability.

```

> -c '{"Args":["GetLineage","cheese1"]}'
[{"event_id":"4f2d76fd1e29bb29b136f47fbc967a015e8f338c88498e84bcb69951e8ff8f3","input_products":["milk1"],"output_products":["cheese1"],"actor":"Org2MSP","event_type":"Processing","timestamp":"2025-11-04T11:10:31Z"}, {"event_id":"10d98d900e676d83ccc56beae029d65d2e47f0d81cdf5227c6b822e124971339","input_products":[],"output_products":["milk1"],"actor":"Org1MSP","event_type":"Production","timestamp":

```

Figure 7. GetLineage() test result.

Figure 8 shows the input parameters and the test result of invoking the RateByBuyer() function. As shown in the figure, the invoked function is RateByBuyer(), where the parameters “DairyFarm”, “5”, and “0.7” represent the enterprise being rated (DairyFarm), the buyer’s score (5), and the weight of the buyer’s rating (0.7), respectively. The figure also shows that the function was executed successfully. This figure demonstrates that buyer evaluations can be correctly submitted and recorded on the blockchain, ensuring that trust scores are dynamically updated.

```

> -c '{"Args":["RateByBuyer","DairyFarm","5","0.7"]}' \
> --waitForEvent \
> --peerAddresses localhost:9051 \
> --tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $(PWD)/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
2025-11-04 20:28:08.775 JST 0001 INFO [chaincodeCmd] ClientWait -> txid [9975ead33a883be77c12640015492c2fb3cc4c09d2e459ed0483b7666aa29955] committed with status (VALID) at localhost:7051
2025-11-04 20:28:08.780 JST 0002 INFO [chaincodeCmd] ClientWait -> txid [9975ead33a883be77c12640015492c2fb3cc4c09d2e459ed0483b7666aa29955] committed with status (VALID) at localhost:9051
2025-11-04 20:28:08.780 JST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200

```

Figure 8. RateByBuyer() test result.

Figure 9 shows the input parameters and the test result of invoking the RateByGovernment() function. As shown in the figure, the invoked function is RateByGovernment(), where the parameters “DairyFarm”, “4”, and “0.7” represent the enterprise being rated (DairyFarm), the regulator’s score (4), and the weight of the buyer’s rating (0.7), respectively. The figure also shows that the function was executed successfully. This figure

demonstrates that regulatory evaluations can be correctly submitted and incorporated into the trust calculation, improving the reliability of enterprise trust scores.

```
c '["A> -c '["Args":["RateByGovernment","DairyFarm","4","0.7"]]' \
> --waitForEvent \
erAddr> --peerAddresses localhost:7051 \
> --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
> --peerAddresses localhost:9051 \
> --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt
2025-11-04 20:35:44.074 JST 0001 INFO [chaincodeCmd] ClientWait -> txid [22b2abb62574b6d76b305f687925d0b3df12430f54ad11980c618fad908c8b42] committed with status (VALID) at localhost:9051
2025-11-04 20:35:44.079 JST 0002 INFO [chaincodeCmd] ClientWait -> txid [22b2abb62574b6d76b305f687925d0b3df12430f54ad11980c618fad908c8b42] committed with status (VALID) at localhost:7051
2025-11-04 20:35:44.079 JST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Figure 9. RateByGovernment() test result.

Figure 10 shows the input parameters and the test result of invoking the QueryMerchantLatestScores() function. As shown in the figure, the invoked function is QueryMerchantLatestScores(), where the parameter "DairyFarm" indicates that the enterprise being queried is DairyFarm. The figure also shows the trust score query results, which record the enterprise's buyer average score, regulator score, composite score, grade, and ban status. The composite score is calculated based on the previously recorded buyer and government ratings. This figure demonstrates that the system can correctly query and present the latest trust status of an enterprise, verifies the successful execution of the RateByBuyer() and RateByGovernment() functions and confirms that the trust mechanism operates as designed.

```
'["A> -c '["Args":["QueryMerchantLatestScores","DairyFarm"]]'
{"buyer_average":0,"government_score":4,"composite_score":4.7,"grade":"S","is_banned":false}
```

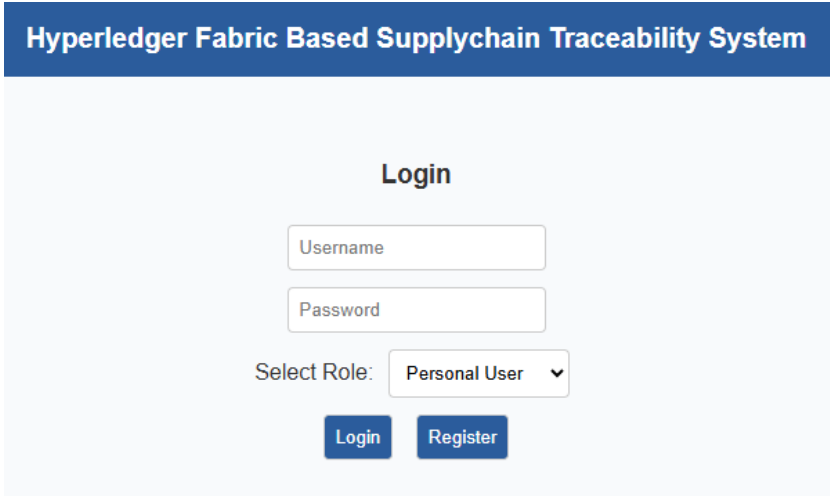
Figure 10. QueryMerchantLatestScores() test result.

We conducted comprehensive functional testing of the chaincode to verify that all core functions operated correctly. The results demonstrate that the system can accurately handle product creation, processing, traceability, and trust scoring operations, confirming both the correctness of the chaincode design and the effectiveness of the overall system functionality.

5. User Interface

To make the system more complete and to visualize the effects of chaincode invocations more conveniently, a simple user interface was designed. The following figures illustrate the layout of the user interface.

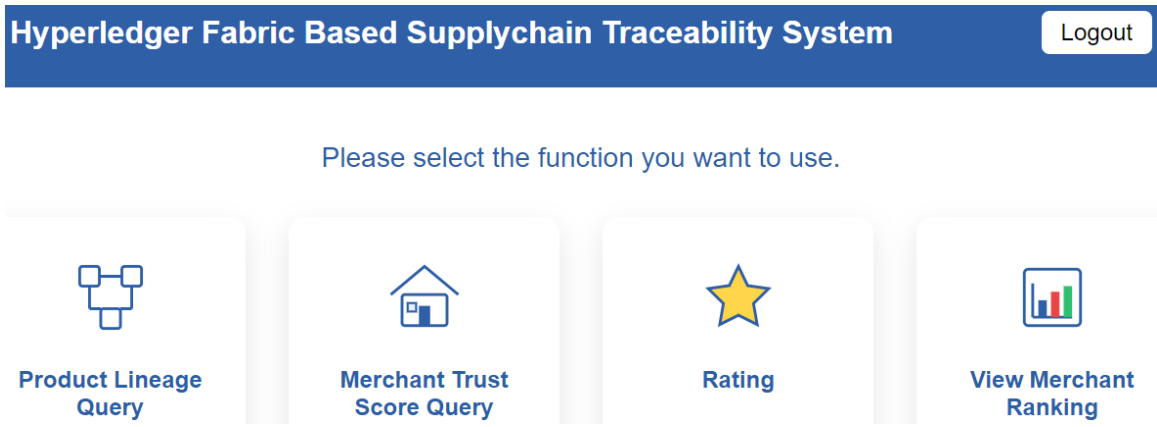
Figure 11 shows the user login interface. As illustrated, users can choose to log in either as personal users or enterprise users. If they have not registered yet, they can complete the registration first; registered users can log in by entering their account ID and password. The account and password information are stored in an off-chain database. After logging in, users are directed to the function selection interface, where the available functions differ between personal and enterprise users. The function selection interface for personal users is introduced below as an example.



The login interface features a dark blue header with the text "Hyperledger Fabric Based Supplychain Traceability System". Below the header, the word "Login" is centered. There are two input fields: "Username" and "Password". Below these is a "Select Role:" dropdown menu currently showing "Personal User". At the bottom are two buttons: "Login" and "Register".

Figure 11. User Login Interface.

Figure 12 shows the function selection interface for personal users. As illustrated, users can choose among functions such as traceability information query, rating query, and merchant rating. When a user clicks on a specific function, the backend invokes the corresponding chaincode function and returns data in JSON format. After data parsing, formatting, and other processing, the results are presented on the interface in an intuitive and user-friendly manner. The following section takes the “traceability information query” and “rating query” functions as examples to provide further explanation of the functional interface.



The function selection interface has a dark blue header with the text "Hyperledger Fabric Based Supplychain Traceability System" and a "Logout" button on the right. Below the header, the text "Please select the function you want to use." is centered. There are four function cards: "Product Lineage Query" (with a chain diagram icon), "Merchant Trust Score Query" (with a house icon), "Rating" (with a star icon), and "View Merchant Ranking" (with a bar chart icon).

Figure 12. Personal User Function Selection Interface.

Figure 13 shows the user interface of the product traceability query function selected in Figure 12. As illustrated, after entering a product ID and clicking the “Search” button, the system displays the entire process of the product from production to consumption in the form of a chain diagram, intuitively showing each stage including producer, processor, retailer, and consumer. By clicking on a specific stage, users can also view detailed information. The displayed results are consistent with those obtained in the functional tests in the previous section, which also verifies the effectiveness of the chaincode functions.

Figure 14 shows the user interface of the merchant trust score query function selected in Figure 12. As illustrated, after entering the merchant’s name and clicking the “Search” button, the system displays the merchant’s trust-related scores, including buyer score, government score, composite score, grade, and current status. The displayed results are consistent with those obtained in the functional tests in the previous section, which also confirms the effectiveness of the chaincode functions.

Product Lineage Query

Enter Product ID to view its production and circulation history.

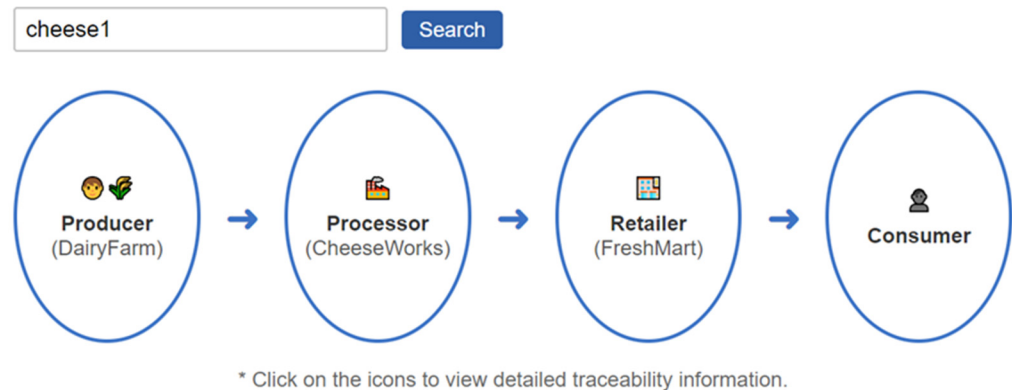


Figure 13. Product Traceability Query Function Interface.

Merchant Trust Score Query

Enter Merchant Name to view reputation data.

The diagram shows the merchant trust score query interface. It includes a search bar with the text 'DairyFarm' and a 'Search' button. Below the search bar, a light blue box displays the search results for 'DairyFarm':

- Merchant:** DairyFarm
- Buyer Score:** 5
- Government Score:** 4
- Composite Score:** 4.7
- Grade:** S
- Status:** ☒ Active

Figure 14. Merchant Trust Score Query Function Interface.

This section introduced the user interface of our system. At present, only a simple interface has been implemented for functional verification and demonstration of chaincode execution results. In future work, we plan to develop a more user-friendly and interactive interface with enhanced usability, aiming to evolve it into a fully functional enterprise-level blockchain application suitable for real-world deployment.

6. Performance Test and Analysis

Performance analysis is crucial for evaluating the efficiency, reliability, and scalability of the system in real environments. It helps identify bottlenecks, optimize resource utilization, and ensure that the system can meet real-world application demands.

6.1. Test Environment

The system was evaluated using Hyperledger Caliper [36], a benchmarking tool designed for measuring the performance of blockchain networks such as Hyperledger Fabric. Hyperledger Caliper provides metrics such as throughput and latency, enabling objective evaluation of system performance under different workloads [36]. Our experimental environment was built on a personal computer equipped with 4 CPU cores, an Intel Core i7-10510U processor (1.80 GHz), and 12 GB of memory. The system ran Linux Ubuntu 22.04 through the Windows Subsystem for Linux (WSL).

6.2. Performance Measurement

For performance measurement, the following two key indicators are selected: throughput and latency.

- Throughput: It reflects the number of transactions successfully processed per second, indicating system capacity.
- Latency: It measures the time from transaction submission to confirmation, representing system responsiveness.

Together, they provide a comprehensive view of efficiency under different workloads.

6.3. Benchmark Setup

Before conducting performance testing, we set up a benchmark workspace consisting of three parts: network configuration, workload, and benchmark. Network configuration is responsible for setting basic network parameters, including channel configuration and other related information. Workload defines the specific way of interacting with the chaincode. Benchmark is used to set testing parameters during the performance test process.

To get different results during performance test, our benchmark configuration includes following parameters [36]:

- Worker Numbers: It represents the number of concurrent client threads sending transactions, reflecting system concurrency capability. We set it to 5, 25, and 50 to evaluate scalability under different parallel workloads.
- Number of Transactions (txNumber): It indicates the total transactions submitted during each test, reflecting workload size. We set it to 1000, 2000, and 5000 to compare performance under varying transaction volumes.
- Transactions Per Second (TPS): It defines the rate of transaction submission per second, reflecting input pressure. We set it to 50, 80, and 100 to evaluate system responsiveness under different input loads.

6.4. Evaluation Result

We mainly tested four chaincode functions: CreateProduct(), ReadProduct(), RateBy-Buyer(), and DeleteProduct(). Unlike the function test, in the performance test we selected only these four functions to represent the four basic CRUD operations (Create, Read, Update, and Delete). The other functions also fall into one of these four categories, so their tests were omitted.

6.4.1. CreateProduct()

To test the impact of total transactions and TPS on system performance, we fixed the number of workers at 5 and varied the total transaction volume and TPS, with results shown in Table 5. As TPS increases, both send rate and throughput rise, demonstrating the system's ability to process higher workloads efficiently under increased input pressure. In addition, latency decreases when TPS grows, which can be explained by Hyperledger Fabric's block batching mechanism and parallel validation and endorsement. With higher TPS, blocks are filled more quickly and verified in batches, reducing per-transaction latency. When TPS is fixed, increasing the total number of transactions causes no significant fluctuation in send rate or throughput, indicating that system performance remains stable over larger transaction loads and is mainly constrained by transaction rate rather than transaction volume.

To evaluate the impact of worker numbers, we fixed the total transactions at 2000 and TPS at 80, and varied the number of workers, with results shown in Table 6. When the worker number increased from 5 to 25, both send rate and throughput improved significantly while latency remained almost unchanged, indicating that the system's parallel processing capacity was fully utilized with more concurrent threads, without greatly affecting responsiveness. However, when the number of workers further increased to 50, send rate and throughput declined while latency rose sharply. This can be attributed

to scheduling overhead and resource contention caused by excessive clients. The results demonstrate that moderate growth in worker numbers enhances system scalability, but beyond a certain threshold, performance degrades due to resource saturation.

Table 5. Performance of CreateProduct(): Fixed N. of Workers, Different txNumber and TPS.

N. of Workers	txNumber	TPS	Send Rate	Throughput	Avg Latency
5	1000	50	15.7	15.3	0.46
		80	19.7	18.9	0.36
		100	25.3	24.2	0.31
5	2000	50	16.2	15.6	0.41
		80	20.4	20.0	0.36
		100	24.1	23.8	0.33
5	5000	50	18.2	17.9	0.42
		80	23.9	23.7	0.35
		100	27.7	27.4	0.30

Table 6. Performance of CreateProduct(): Fixed txNumber and TPS, Different N. of Workers.

txNumber	TPS	N. of Workers	Send Rate	Throughput	Avg Latency
2000	80	5	20.4	20.0	0.36
		25	40.9	40.5	0.37
		50	31.0	30.0	0.65

6.4.2. ReadProduct()

The setup is identical to the CreateProduct() tests. Table 7 shows the results of testing ReadProduct() function with a fixed number of workers. As illustrated in Table 7, when TPS increases, both send rate and throughput rise slightly, indicating that higher submission rates can still be efficiently absorbed by the system. Similarly, increasing the total number of transactions leads to modest gains in throughput, showing that the system performance remains stable under larger workloads. Regardless of TPS or transaction volume, throughput consistently stays at a high level, while latency remains extremely low at 0.02 s, indicating that the execution of ReadProduct() function is very efficient. This is because ReadProduct() is a read-only operation that does not require endorsement or state updates, allowing the system to process queries rapidly with minimal computational overhead.

Table 7. Performance of ReadProduct(): Fixed N. of Workers, Different txNumber and TPS.

N. of Workers	txNumber	TPS	Send Rate	Throughput	Avg Latency
5	1000	50	176.1	175.8	0.02
		80	206.5	206.0	0.02
		100	225.1	224.4	0.02
5	2000	50	307.7	307.3	0.02
		80	296.6	296.2	0.02
		100	325.7	325.2	0.02
5	5000	50	347.2	347.1	0.02
		80	350.8	350.5	0.02
		100	365.5	365.3	0.02

Table 8 presents the results of the ReadProduct() function under fixed transaction volume and TPS. As shown, when the number of workers increased to 25, both send rate

and throughput dropped sharply, while latency rose significantly. This is mainly due to excessive client scheduling overhead and resource contention, which reduce efficiency. However, when the number of workers increased further to 50, send rate and throughput partially recovered and latency decreased markedly. This can be explained as higher concurrency compensating for part of the scheduling overhead, thereby improving resource utilization and restoring throughput. Hence, for query operations, five workers provide the most favorable performance; adding a moderate number of workers reduces efficiency, but a large increase in workers may allow performance to partially recover.

Table 8. Performance of ReadProduct(): Fixed txNumber and TPS, Different N. of Workers.

txNumber	TPS	N. of Workers	Send Rate	Throughput	Avg Latency
2000	80	5	296.6	296.2	0.02
		25	55.9	55.9	0.26
		50	103.2	103.1	0.04

6.4.3. RateByBuyer()

The setup is identical to the CreateProduct() tests. Table 9 shows the results of the RateByBuyer() function with a fixed number of workers. As TPS increases, both throughput and latency improve slightly, but the gains are limited, especially at TPS 100, suggesting that system performance may be approaching saturation. The likely cause is hardware constraints of the experimental machine, which introduce a performance bottleneck. When the total number of transactions increases, throughput also rises modestly while remaining stable overall, indicating that the system can consistently handle update operations under heavier workloads. These results suggest that RateByBuyer(), as an update function, maintains stable performance even when transaction volume grows, though its scalability is bounded by underlying hardware capacity.

Table 9. Performance of RateByBuyer(): Fixed N. of Workers, Different txNumber and TPS.

N. of Workers	txNumber	TPS	Send Rate	Throughput	Avg Latency
5	1000	50	22.5	21.1	0.35
		80	27.6	26.8	0.26
		100	31.2	30.9	0.26
5	2000	50	22.0	21.0	0.34
		80	26.5	26.2	0.29
		100	32.5	32.2	0.25
5	5000	50	24.2	25.9	0.30
		80	30.7	30.4	0.27
		100	32.9	32.6	0.26

Table 10 presents the results of the RateByBuyer() function under fixed transaction volume and TPS. When the number of workers increased to 25, both send rate and throughput improved significantly, while latency rose slightly, indicating that the system can effectively handle a moderate increase in concurrency. However, when the worker number increased further to 50, throughput dropped back to the level observed with 5 workers, and latency increased sharply. This is likely caused by scheduling overhead and resource contention among excessive clients. These results suggest that a moderate increase in concurrent threads can enhance performance, but too many may lead to diminishing returns. For

update operations, 25 workers appear to be an optimal setting, achieving the highest throughput with only a modest and acceptable rise in latency.

Table 10. Performance of RateByBuyer(): Fixed txNumber and TPS, Different N. of Workers.

txNumber	TPS	N. of Workers	Send Rate	Throughput	Avg Latency
2000	80	5	26.5	26.2	0.29
		25	40.3	39.4	0.33
		50	28.5	28.1	0.84

6.4.4. DeleteProduct()

The setup is identical to the CreateProduct() tests. Table 11 presents the results of the DeleteProduct() function with a fixed number of workers. Similarly to the RateByBuyer() function, as TPS increases, both throughput and latency rise slightly, but the improvement is limited. This may be explained by Hyperledger Fabric's batch processing mechanism enhancing efficiency, while the hardware limitations of the experimental machine introduce performance bottlenecks. When the total number of transactions increases, throughput also shows a modest improvement while remaining stable overall, indicating that the system can consistently handle deletion operations under heavier workloads. Overall, delete operations behave similarly to update operations: the system maintains stable performance as workload grows, but its scalability under higher TPS is constrained by hardware capacity.

Table 11. Performance of DeleteProduct(): Fixed N. of Workers, Different txNumber and TPS.

N. of Workers	txNumber	TPS	Send Rate	Throughput	Avg Latency
5	1000	50	20.2	19.7	0.35
		80	25.4	24.9	0.31
		100	28.5	27.8	0.27
5	2000	50	24.3	24.2	0.35
		80	28.9	28.5	0.28
		100	31.5	30.9	0.28
5	5000	50	25.2	24.9	0.30
		80	31.4	31.0	0.27
		100	32.0	31.5	0.26

Table 12 shows the results of the DeleteProduct() function under fixed transaction volume and TPS. The results are similar to the RateByBuyer() function: as the number of workers increases, send rate and throughput first rise and then fall, while latency continuously increases. This pattern reflects that a moderate increase in concurrency can fully exploit the system's parallel processing capability, but excessive concurrency leads to intensified resource contention. It can be observed that, similar to update operations, the optimal configuration for delete operations is around 25 workers.

Table 12. Performance of DeleteProduct(): Fixed txNumber and TPS, Different N. of Workers.

txNumber	TPS	N. of Workers	Send Rate	Throughput	Avg Latency
2000	80	5	28.9	28.5	0.28
		25	36.2	35.9	0.33
		50	30.4	29.9	0.56

7. Discussion

In this section, we will discuss the strengths and weaknesses of the proposed system. Pros highlight its core advantages, including distributed architecture, privacy protection, and trust mechanisms and other advantages. On the other hand, Cons point out deficiencies that need improvement, such as system performance and architectural complexity.

7.1. Pros of the Proposed System

Compared with traditional centralized databases, the proposed system leverages blockchain technology to achieve decentralization and immutability. Through its distributed architecture, it eliminates reliance on a central authority and reduces the risk of single-point failures, while immutability ensures that once data is recorded on the ledger, it cannot be altered.

Unlike public blockchains such as Ethereum [37], this system is built on Hyperledger Fabric, ensuring that only authorized organizations can join the network. In addition, the system employs a multi-channel architecture that separates private data into different channels, with access control mechanisms introduced in private channels to enforce strict data isolation and enhance privacy protection.

Compared with other multi-blockchain supply chain traceability studies, the system introduces a trust mechanism and involves regulatory authorities on-chain to provide effective supervision over enterprises. This enables stakeholders to collectively evaluate data reliability, mitigates the GIGO problem, discourages dishonest behavior, and motivates enterprises to upload authentic data, thereby improving the overall credibility of supply chain records.

7.2. Cons of the Proposed System

Despite its advantages, our system also has several limitations. Due to the inclusion of operations such as consensus mechanisms and organizational endorsements, transaction processing speed may be lower compared with traditional database systems, making it less suitable for high-frequency trading environments. In addition, maintaining a Hyperledger Fabric network requires considerable operational effort, and system updates or expansions can be more complex than conventional solutions. Another limitation is related to data storage: in the current design, all traceability data are stored directly on the blockchain, which increases on-chain storage overhead and may lead to scalability and performance challenges as the system grows. This design choice can impose significant storage pressure on peer nodes, especially in long-term or large-scale deployment scenarios. Against this background, the proposed system is more suitable for application scenarios in which the volume of traceability data is relatively limited, such as small- to medium-scale supply chains or products with concise and event-driven traceability records. In such contexts, storing traceability information directly on the blockchain can still provide strong data integrity and transparency guarantees without causing excessive storage or performance burdens.

In practice, when storing all data entirely on the blockchain becomes impractical, a hybrid storage approach is commonly adopted, in which only cryptographic hash values (fingerprints) of the data are recorded on the blockchain, while the actual data are stored off-chain, such as in cloud-based storage systems [38]. This hybrid on-chain/off-chain design has become a mainstream solution to mitigate on-chain storage pressure [39].

Another drawback lies in endpoint verification: while the system records supply chain processes, it lacks a “one item, one certificate” mechanism to guarantee product authenticity. As a result, consumers can view supply chain records but cannot fully prove that “this product is the one”, counterfeit goods could share the same traceability page, thereby limiting the system’s effectiveness in preventing fraud. It should be noted that the

endpoint verification issue is not a limitation unique to the proposed system, but rather a common challenge faced by most blockchain-based supply chain traceability systems. This challenge stems from the inherent gap between physical products in the real world and their digital representations on the blockchain [4]. The issue can be mitigated by integrating complementary technologies, such as non-fungible tokens (NFTs) and RFID tags, to strengthen the binding between physical items and their on-chain identities.

Despite this drawback, the proposed system remains suitable for supply chain traceability scenarios in which endpoint verification of individual physical items is not the primary requirement. In particular, it is well suited for application contexts that focus on inter-organizational collaboration, process compliance auditing, and regulatory supervision. For example, in food or agri-food supply chains, the system can be used to record key operational processes such as raw material sourcing, processing, transportation, and distribution, thereby supporting information sharing among enterprises and post-incident compliance auditing by regulatory authorities. In such scenarios, the core value of the system lies in ensuring traceability, auditability, and accountability of supply chain processes and participant behaviors, rather than in the direct verification of the authenticity of individual physical products at the consumer end.

7.3. Comparison with Other Blockchain-Based Systems

To highlight the characteristics and advantages of the proposed system, it is compared with several related works that are based on Hyperledger Fabric. The result is shown in Table 13.

Table 13. Comparison with Other Hyperledger Fabric-based Systems.

	Study [28]	Study [23]	Study [33]	Our Proposal
Domain	Food Supply Chain	Supply Chain	IoV	Food Supply Chain
Underlying Technology	Hyperledger Fabric	Hyperledger Fabric	Hyperledger Fabric	Hyperledger Fabric
Architecture	Permissioned network	Multi-blockchain architecture (one consortium blockchain + multiple private blockchains)	Multi-channel architecture (data partitioning and management)	Same as [33]
Privacy Protection	Access control + encryption	Privacy stored in private blockchains + access control	Channel separation	Channel separation + access control
Data Reliability Assurance	Tamper-Resistance (a fundamental property of blockchain)	Same as [28]	Same as [28]	Tamper-Resistance + Trust-value-based incentive for data submission
Incentive Design	None	None	None	Incentives based on trust scores
Evaluation Method	Real-world deployments (e.g., Walmart)	Chaincode functional testing + Hyperledger Caliper performance evaluation	Same as [23]	Same as [23]

As shown in Table 13, the proposed system and the compared systems share the common feature of adopting Hyperledger Fabric as the underlying platform. The main differences lie in privacy protection and data trustworthiness assurance. Except for the study [28] (IBM Food Trust), other systems employ multi-chain or multi-channel architecture to achieve data separation and protect private information. Since the internal architecture of IBM Food Trust is not open-sourced, it remains unclear whether it adopts a multi-chain or multi-channel structure; however, it is generally considered to rely on a permissioned network combined with access control to protect sensitive data. Although the study [23] proposes a multi-chain architecture, it does not physically isolate data using multiple channels in its implementation, but instead only applies logical isolation through different peer nodes in one channel. The study [33] adopts a multi-channel structure for physical data isolation, but its data exchange mechanism is not sufficiently comprehensive. In contrast, the proposed system introduces channel-level access control to enable more secure and comprehensive data exchange in the multi-channel architecture. More importantly, the key distinction of the proposed system lies in data trustworthiness assurance. Other systems generally assume that data are correct before being recorded on-chain, thereby overlooking the GIGO problem. To address this issue, the proposed system introduces trust score and incentive mechanisms to encourage supply chain participants to submit more reliable data. Overall, compared with other systems, this work simultaneously improves privacy protection and data trustworthiness, addressing both aspects at the same time.

In addition to Hyperledger Fabric-based systems, some supply chain traceability solutions have been developed on public blockchain platforms such as Ethereum, many of which are combined with off-chain storage technologies like IPFS [25]. In such approaches, large-scale traceability data are typically stored on IPFS, while cryptographic hash pointers and transaction metadata are recorded on the public blockchain, thereby ensuring data integrity while reducing on-chain storage overhead [25]. However, public-chain-based traceability systems are generally designed for open participation environments, and their privacy protection mainly relies on encryption and application-layer access control. As a result, native mechanisms for fine-grained data isolation are relatively limited, which leads to certain constraints in meeting supply chain requirements for controllable participation and data isolation [37]. In contrast, the proposed system operates in a permissioned blockchain environment, where explicit identity management and channel-level data isolation mechanisms enable more effective privacy protection for multi-organizational collaboration. In addition, public-chain-based systems usually require transaction fees for each operation, and such fees are typically paid using cryptocurrencies with real monetary value, introducing continuous and non-negligible operational costs in high-frequency supply chain applications [37]. By contrast, the proposed system runs in a consortium blockchain environment and does not rely on transaction fee mechanisms, making it more suitable for long-term and stable deployment. Moreover, public blockchains may experience network congestion under high load, leading to increased transaction confirmation latency and reduced system throughput [37]. In contrast, by leveraging a controlled participation environment and deterministic execution processes, the proposed system can provide more stable system performance, better aligning with the operational requirements of real-world supply chain applications. Finally, from a regulatory perspective, public-chain-based traceability systems often require extensive off-chain coordination and external institutional support due to open participation, weak binding between on-chain identities and real-world entities, and the lack of unified governance and access control mechanisms [37]. In comparison, the proposed system, built on a permissioned blockchain architecture, features clearly identified participants and predefined governance rules, making it more suitable

for achieving effective supervision and accountability in food supply chain traceability scenarios that require regulatory involvement.

7.4. Comparison with the Previous Work

This study can be regarded as an extension and improvement of a prior work [33], which was conducted within the same research group and shares a common research background. Although the two studies are closely related in terms of research lineage, they differ in application contexts, and the present work introduces new system design considerations and functional enhancements. Therefore, it is necessary to explicitly clarify the relationship and differences between this study and the previous work [33].

As discussed in the Literature Review, the study [33] proposed a Hyperledger Fabric-based multi-channel architecture to enhance secure and efficient data exchange in IoV. Similarly, the present study also adopts a multi-channel architecture based on Hyperledger Fabric, where different types of data are managed in separate channels. This design enables physical data separation and contributes to improved privacy protection in a multi-organizational environment.

Compared with the study [33], the role of sub-channels in the present work remains largely consistent, as they are primarily used to manage privacy-sensitive data. However, due to the difference in application contexts, the role of the main channel differs between the two studies. In the study [33], the main channel mainly serves as an intermediary for cross-channel data exchange. In contrast, in the proposed system targeting food supply chain traceability, the main channel not only supports cross-channel data exchange but also undertakes supply chain traceability functions. More importantly, to address the more severe GIGO problem in food supply chain traceability scenarios, a trust and incentive mechanism is further introduced in the main channel to encourage blockchain participants to submit more truthful and reliable data.

Another important difference lies in the cross-channel data exchange mechanism. In the study [33], the data exchange process among different organizations on the main channel is not sufficiently specified. For ease of explanation, a simplified system example is illustrated in Figure 15, where the peer nodes of Org1, Org2, and Org3 participate in the main channel A, while they separately participate in sub-channels B, C, and D. In the design of the study [33], only Org1 is able to perform cross-channel queries to access data in channel B. However, after Org1 retrieves data from channel B by invoking the cross-channel chaincode on channel A, how the returned results are further shared with Org2 or Org3 is not clearly defined, resulting in an incomplete data exchange mechanism. In contrast, in the proposed design, Org1, Org2, and Org3 are all allowed to invoke the cross-channel chaincode deployed on channel A to query data from channel B. At the same time, an access control list (ACL) is maintained within channel B, such that only authorized organizations are permitted to access the data. This ACL can be dynamically updated; for example, Org2 is able to access data in channel B only after authorization is granted and is denied access otherwise. The same mechanism applies to channels C and D. Through this design, the ambiguity in inter-organizational data sharing on the main channel is fundamentally addressed, enabling a more secure, controllable, and well-defined cross-channel data exchange process.

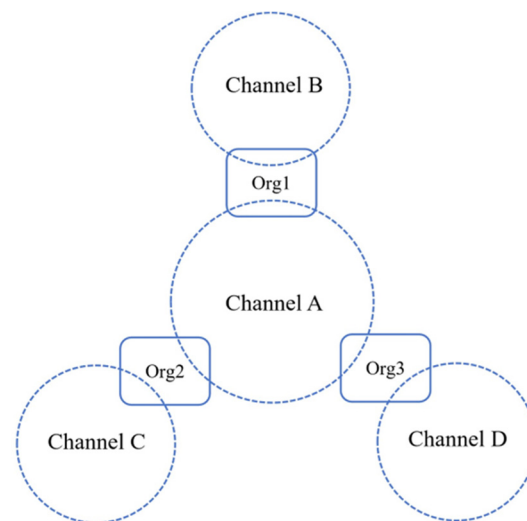


Figure 15. System example diagram.

8. Conclusions and Future Work

This paper presents a Hyperledger Fabric-based food supply chain traceability system, which leverages blockchain to achieve decentralization, data immutability, enhanced privacy protection, and improved credibility of supply chain records. The main contribution of this system lies in its multi-channel architecture, which separates private data into different channels and enforces access control mechanisms to enhance privacy protection. At the same time, regulatory authorities are introduced on-chain, and a trust mechanism is implemented through chaincode to mitigate the GIGO problem and improve the credibility of on-chain data. Furthermore, we deployed the system, tested chaincode functions, and analyzed performance under different conditions. The results confirmed the stability and reliability of the system.

This system still has several aspects that require further research and improvement, particularly in terms of on-chain storage overhead and endpoint verification. In response to the issue of on-chain storage overhead, a hybrid on-chain and off-chain storage approach can be adopted to alleviate on-chain storage pressure, for example, by integrating IPFS to store large-scale data while recording only their hash references on the blockchain. At the same time, the impacts of introducing IPFS on the multi-channel system in terms of privacy protection, performance, scalability, and other aspects will be further explored. In response to the issue of endpoint verification, NFT technology [40] can be integrated to strengthen authenticity verification on the consumer side by providing a unique digital certificate for each product, thereby achieving “one item, one certificate” and preventing counterfeit goods from sharing the same traceability page. Finally, large-scale testing in real-world scenarios should be conducted to evaluate the system’s scalability, robustness, and adaptability in practical food supply chain environments.

Author Contributions: Conceptualization, Z.Z., Y.F. and K.S.; methodology, Z.Z. and Y.F.; software, Z.Z.; validation, Z.Z.; formal analysis, Z.Z. and Y.F.; investigation, Z.Z. and Y.F.; writing—original draft preparation, Z.Z.; writing—review and editing, Y.F. and K.S.; visualization, Z.Z.; supervision, Y.F. and K.S.; project administration, Z.Z., Y.F. and K.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Aung, M.M.; Chang, Y.S. Traceability in a food supply chain: Safety and quality perspectives. *Food Control* **2014**, *39*, 172–184. [CrossRef]
2. Dabbene, F.; Gay, P.; Tortia, C. Traceability issues in food supply chain management: A review. *Biosyst. Eng.* **2014**, *120*, 65–80. [CrossRef]
3. Oh, S.E.; Kim, J.-H.; Kim, J.-Y.; Ahn, J.-H. Food Safety Distribution Systems Using Private Blockchain: Ensuring Traceability and Data Integrity Verification. *Foods* **2025**, *14*, 1405. [CrossRef]
4. Cordeiro, M.; Ferreira, J.C. Beyond Traceability: Decentralised Identity and Digital Twins for Verifiable Product Identity in Agri-Food Supply Chains. *Appl. Sci.* **2025**, *15*, 6062. [CrossRef]
5. Duan, Y.; Zhu, Q. Blockchain empowerment: Enhancing consumer trust and outreach through supply chain transparency. *Int. J. Prod. Res.* **2024**, *63*, 5358–5382. [CrossRef]
6. Sri Vigna Hema, V.S. Blockchain implementation for food safety in supply chain: A review. *Compr. Rev. Food Sci. Food Saf.* **2024**, *23*, e70002. [CrossRef]
7. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. White Paper. 2008. Available online: <https://bitcoin.org/en/bitcoin-paper> (accessed on 1 February 2026).
8. Crosby, M.; Pattanayak, P.; Verma, S.; Kalyanaraman, V. Blockchain technology: Beyond bitcoin. *Appl. Innov. Rev.* **2016**, *2*, 6–19.
9. Zheng, Z.; Xie, S.; Dai, H.; Chen, X.; Wang, H. An overview of blockchain technology: Architecture, consensus, and future trends. In *2017 IEEE International Congress on Big Data (BigData Congress)*; IEEE: Piscataway, NJ, USA, 2017; pp. 557–564.
10. Androulaki, E.; Barger, A.; Bortnikov, V.; Cachin, C.; Christidis, K.; Caro, A.D.; Enyeart, D. Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. In *EuroSys '18: Proceedings of the Thirteenth EuroSys Conference*; ACM: New York, NY, USA, 2018; pp. 1–15. [CrossRef]
11. Alsallut, A.Y.; Salamah, R.A.; Abusamra, A.A. Exploratory study on Hyperledger Fabric framework: Food supply chain as a case study. *Int. J. Eng. Manuf.* **2023**, *13*, 11–19. [CrossRef]
12. Cachin, C. Architecture of the Hyperledger Blockchain Fabric. In *Workshop on Distributed Cryptocurrencies and Consensus Ledgers (DCCL)*; IBM Research-Zurich: Ruschlikon, Switzerland, 2016; Volume 4, pp. 1–4.
13. Gorenflo, C.; Lee, S.; Golab, L.; Keshav, S. FastFabric: Scaling Hyperledger Fabric to 20,000 Transactions per Second. *Int. J. Netw. Manag.* **2019**, *30*, e2099. [CrossRef]
14. Thakkar, P.; Nathan, S.; Viswanathan, B. Performance benchmarking and optimizing Hyperledger Fabric blockchain platform. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*; IEEE: Piscataway, NJ, USA, 2018.
15. Baliga, A. Understanding blockchain consensus models. In *Persistent Systems Technical Report*; Persistent Systems: Pune, India, 2018.
16. El Hajji, M.; Es-saady, Y.; Ait Addi, M.; Antari, J. Optimization of agrifood supply chains using Hyperledger Fabric blockchain technology. *Comput. Electr. Eng.* **2024**, *227*, 109503. [CrossRef]
17. Sharma, P.K.; Chen, M.Y.; Park, J.H. A software defined fog node based distributed blockchain cloud architecture for IoT. *IEEE Access* **2018**, *6*, 115–124. [CrossRef]
18. Abeyratne, S.A.; Monfared, R.P. Blockchain ready manufacturing supply chain using distributed ledger. *Int. J. Res. Eng. Technol.* **2016**, *5*, 1–10. [CrossRef]
19. Powell, W.; Foth, M.; Cao, S.; Natanelov, V. Garbage in garbage out: The precarious link between IoT and blockchain in food supply chains. *J. Ind. Inf. Integr.* **2022**, *25*, 100261. [CrossRef]
20. Casino, F.; Kanakaris, V.; Dasaklis, T.K.; Moschuris, S.; Rachaniotis, N.P. Blockchain-based food supply chain traceability: A case study in the dairy sector. *Int. J. Prod. Res.* **2021**, *59*, 5758–5770. [CrossRef]
21. Ellahi, R.M.; Wood, L.C.; Bekhit, A.E. Blockchain-Driven Food Supply Chains: A Systematic Review for Unexplored Opportunities. *Appl. Sci.* **2024**, *14*, 8944. [CrossRef]
22. Galvez, J.F.; Mejuto, J.C.; Simal-Gandara, J. Future challenges on the use of blockchain for food traceability analysis. *TrAC Trends Anal. Chem.* **2018**, *107*, 222–232. [CrossRef]
23. Ding, Q.; Gao, S.; Zhu, J.; Yuan, C. Permissioned Blockchain-Based Double-Layer Framework for Product Traceability System. *IEEE Access* **2020**, *8*, 174151–174161. [CrossRef]
24. Wang, S.; Yang, M.; Zhang, Y.; Luo, Y.; Ge, T.; Fu, X.; Zhao, W. On Private Data Collection of Hyperledger Fabric. In *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*; IEEE: Piscataway, NJ, USA, 2021; pp. 1–10. [CrossRef]
25. Tian, F. An Agri-Food Supply Chain Traceability System for China Based on RFID & Blockchain Technology. In *2016 13th International Conference on Service Systems and Service Management (ICSSSM)*; IEEE: Piscataway, NJ, USA, 2016; pp. 1–6.

26. Shivala, N.M.; Mahalle, P.; Patil, S.; Bhandari, G.M.; Dere, A.B.; Dethe, C.D.; Jadhav, S.S.; Papat, S.M. A survey on blockchain technology in food supply chain management. *Cureus J. Comput. Sci* **2025**. [CrossRef]
27. Guo, T.; Chen, Y.; Ren, Q.; Li, D.; Bo, W.; Wang, X. Blockchain-Based Trusted Traceability Scheme for Food Quality and Safety. *J. Food Qual.* **2025**, 5914078. [CrossRef]
28. IBM. *IBM Food Trust: An IBM Blockchain Solution for the Complete Food System—Solution Brief 2.0*; IBM Corporation: Armonk, NY, USA, 2018. Available online: <https://www.ibm.com/blockchain/solutions/food-trust> (accessed on 1 February 2026).
29. Marchese, A.; Tomarchio, O. An Agri-Food Supply Chain Traceability Management System based on Hyperledger Fabric Blockchain. In *23rd International Conference on Enterprise Information Systems—Volume 2: ICEIS*; SciTePress: Setúbal, Portugal, 2021; pp. 648–658.
30. Yang, X.; Li, M.; Yu, H.; Wang, M.; Xu, D.; Sun, C. A trusted blockchain-based traceability system for fruit and vegetable agricultural products. *IEEE Access* **2021**, 9, 36282–36297. [CrossRef]
31. Leteane, O.; Ayalew, Y. Improving the Trustworthiness of Traceability Data in Food Supply Chain Using Blockchain and Trust Model. *J. Brit. Blockchain Assoc.* **2024**, 7. [CrossRef] [PubMed]
32. Valapu, S.T.; Saha, A.; Krishnamachari, B.; Menon, V.V.; Guin, U. Reward-based Blockchain Infrastructure for 3D IC Supply Chain Provenance. In *2025 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*; IEEE: Piscataway, NJ, USA, 2025; pp. 370–380.
33. Liu, Y.; Feng, Y.; Sakurai, K. Hyperledger Fabric-Based Multi-Channel Structure for Data Exchange in Internet of Vehicles. *Electronics* **2025**, 14, 572. [CrossRef]
34. Huang, D.; Ma, X.; Zhang, S. Performance Analysis of the Raft Consensus Algorithm for Private Blockchains. *IEEE Trans. Syst. Man Cybern. Syst.* **2020**, 50, 172–181. [CrossRef]
35. Github Contributors. Hyperledger Fabric. Available online: <https://github.com/hyperledger/fabric> (accessed on 1 February 2026).
36. Github Contributors. Hyperledger Caliper. Available online: <https://github.com/hyperledger-caliper/caliper> (accessed on 1 February 2026).
37. Karaduman, Ö.; Gülhas, G. Blockchain-Enabled Supply Chain Management: A Review of Security, Traceability, and Data Integrity Amid the Evolving Systemic Demand. *Appl. Sci.* **2025**, 15, 5168. [CrossRef]
38. Ahmed, W. Blockchain and Big Data Integration in Financial Transparency: Explore How Combining Blockchain with Big Data Improves Transparency in Financial Reporting and Fraud Detection. *J. Econ. Bus. Commer.* **2025**, 2, 129–134. [CrossRef]
39. Theodorakopoulos, L.; Theodoropoulou, A.; Halkiopoulos, C. Enhancing Decentralized Decision-Making with Big Data and Blockchain Technology: A Comprehensive Review. *Appl. Sci.* **2024**, 14, 7007. [CrossRef]
40. Ante, L. Non-fungible token (NFT) markets on the Ethereum blockchain: Temporal development, cointegration and interrelations. *Econ. Innov. New Technol.* **2023**, 32, 1216–1234. [CrossRef]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.